

Code-Layout, Readability and Reusability

Date	27/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	4-space indentation used consistently in all Python files (app.py, flood_edu.ipynb, flood_model.ipynb). PEP8 standards followed throughout.
2	Proper File Structure	Files and folders are logically organized	Yes	Project structured into: templates/ (HTML), static/css/ (main.css), static/js/ (main.js), notebooks, app.py, Procfile, requirements.txt.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like X_train, X_test, y_train, y_test, scaler, xgb_model, feature_names, input_scaled clearly describe their purpose and content.
4	Function / Method Names	Functions are descriptively named	Yes	Functions named evaluate_model(), decisiontree(), randomForest(), KNN(), xgboost_model(), compareModel() clearly describe their role in the pipeline.
5	Code Comments	Inline and block comments explain logic	Yes	All major sections in notebooks are explained using markdown cells. In app.py, inline comments explain model loading, route handling and prediction logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	Each ML model is wrapped in its own function. evaluate_model() is a reusable helper that trains, predicts and plots confusion matrix and ROC curve for any classifier.

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
7	No Redundant Code	Duplicate or unused code is removed	Yes	No duplicate imports or redundant functions. The <code>evaluate_model()</code> function eliminates repetitive evaluation code. All HTML pages share a single <code>main.css</code> stylesheet.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Flask <code>app.py</code> uses <code>try-except</code> block to catch missing or invalid form inputs and returns an error message to the user on the prediction form without crashing the server.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	<code>evaluate_model()</code> Function	Python / Scikit-learn, XGBoost, Matplotlib	Reused for all 4 classifiers: Decision Tree, Random Forest, KNN, XGBoost in <code>flood_model.ipynb</code>	High
2	<code>StandardScaler (transform.save)</code>	Python / Scikit-learn / Joblib	Fitted once on training data in <code>flood_model.ipynb</code> . Reused in <code>app.py</code> at runtime for every prediction request.	High
3	<code>XGBoost Model (floods.save)</code>	Python / XGBoost / Joblib	Trained once in <code>flood_model.ipynb</code> . Loaded and reused in <code>app.py</code> for all real-time flood predictions.	High
4	<code>main.css</code> Stylesheet	CSS3	Shared across all 4 HTML templates: <code>home.html</code> , <code>index.html</code> , <code>chance.html</code> , <code>no_chance.html</code>	High
5	<code>main.js</code> Script	JavaScript	Shared across all 4 HTML templates for rain animation, form validation and probability bar animation.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5 / 5	Project follows a clean MVC-style structure with separate folders for templates, static assets, and Python scripts. <code>Procfile</code> and <code>requirements.txt</code> ensure reproducibility.
Readability	5 / 5	All variables, functions and routes are meaningfully named. Markdown cells in notebooks explain every step. Inline comments in <code>app.py</code> clarify model loading and prediction logic.

Aspect	Rating (1-5)	Comments
Reusability	5 / 5	evaluate_model() eliminates repetitive code for all 4 classifiers. main.css and main.js are shared across all HTML pages. Model and scaler saved separately for flexible reuse.
Documentation / Comments	4 / 5	All notebook cells are documented with markdown headers and inline comments. Flask routes are commented. Minor improvement possible by adding docstrings to all Python functions.
Overall Score	4.75 / 5	The Rising Waters codebase demonstrates excellent layout, readability and reusability. The modular design, consistent naming conventions and shared components make it highly maintainable.